

Conclusions i propostes a futur

Bot Detector — Disseny i Anàlisi d'Algoritmes Avançats



Asignatura: Disseny i Anàlisi d'Algoritmes Avançats

Centro: ENTI · Universitat de Barcelona

Alumno: Víctor Castro Quesada i Oscar Fernandez Andujar

Profesor: Marti de Ferrer

1. Conclusions

Amb aquest projecte hem aconseguit un sistema funcional que detecta atacs de força bruta en logs de servidor. Les conclusions principals són:

- Hem usat quatre tipus d'estructures de dades (cua, diccionari/hash, arbre N-ari i conjunt), cadascuna amb un cost diferent, demostrant que sabem triar l'estructura adequada per a cada problema.
- Hem aplicat recursivitat en dos punts del sistema: la inserció i cerca a l'arbre N-ari, i la detecció d'escaneig de ports.
- Hem usat Programació Orientada a Objectes amb herència i polimorfisme: una classe pare Analizador i dues classes filles que s'utilitzen de forma transparent.
- Hem organitzat el codi seguint el patró MVC (Model, Vista, Controlador), separant clarament la lògica, la pantalla i la coordinació.
- El sistema té un cost lineal $O(N)$, cosa que vol dir que escala bé en augmentar la mida del log.

2. Què hem après

Sobre estructures de dades: Al principi no teníem clar per què necessitàvem un arbre si amb un diccionari ja podíem guardar les IPs. Després, quan vam implementar la detecció de subxarxes, vam entendre la gràcia: l'arbre permet agrupar les IPs per octets i recórrer-les recursivament per trobar patrons. Un diccionari pla no et dona aquesta estructura jeràrquica.

Ens va costar una mica al principi, sobretot amb la funció ``_buscar()`` de l'arbre. El problema era entendre quan parar (el cas base). Un cop vam veure que el cas base és simplement "quan el camí té 3 trossos, estem al nivell de subxarxa i comptem", la resta va sortir sola. La recursivitat de ``_distintos()`` als ports va ser més fàcil perquè és bàsicament recórrer una llista.

Sobre POO i polimorfisme:** Al principi teníem tot el codi en un sol arxiu amb un munt d'`if's` per decidir si la línia era un login fallit o un escaneig de ports. Quan vam separar-ho en classes amb herència, el codi del controlador va quedar molt més net. El moment en què vam veure que el ``for analizador in self.analizadores`` funcionava sense saber quin tipus era cada analitzador, vam entendre de veritat què és el polimorfisme.

Separar el codi en Model, Vista i Controlador ens va ajudar molt a organitzar-nos. Al principi la Vista i el Controlador estaven barrejats (el controlador feia prints directament). Quan vam separar-ho, ens vam adonar que era molt més fàcil canviar com es mostraven les alertes sense tocar la lògica de detecció. També ens va facilitar repartir la feina: un podia treballar al model sense que l'altre hagués d'esperar.

3. Propostes a futur

Si tinguéssim més temps, milloràriem el projecte en aquests punts:

- Optimitzar ``contar_ip()``: Ara cada cop que volem saber quants intents porta una IP, recorrem tota la cua ($O(V)$). Es podria mantenir un diccionari auxiliar que comptés automàticament cada IP quan s'afegeix o s'esborra de la cua. Això faria la consulta $O(1)$ a canvi d'una mica més de memòria.

- Lectura en temps real: Ara el sistema llegeix el log sencer de cop. En un cas real, voldríem que anés llegint les línies noves a mesura que el servidor les escriu (com fa la comanda ``tail -f`` de Linux). Això permetria detectar atacs mentre passen, no després.
- Gràcies al polimorfisme, afegir un nou analitzador és tan fàcil com crear una classe filla nova. Per exemple, un ``AnalizadorWeb`` que detectés peticions sospitoses a pàgines d'administració (401/403 a `/admin` o `/wp-login`), o un analitzador que detectés atacs en horaris de matinada.
- Ara les IPs bloquejades es perden quan el programa acaba. Es podria guardar la llista en un fitxer JSON perquè el sistema recordi els bloquejos encara que es reiniciï.
- Interfície gràfica: El sistema funciona per terminal, que és suficient per a un projecte acadèmic, però una interfície visual (per exemple amb `tkinter`) faria que fos més fàcil d'usar i de presentar.

- **Guardar les IPs bloquejades en un fitxer:** perquè el sistema recordi els bloquejos encara que es reiniciï.
- **Interfície gràfica:** una pantalla visual en comptes de text per consola facilitaria l'ús del sistema.
- **Integració amb feeds d'intel·ligència d'amenaces:** connectar el sistema amb llistes públiques d'IPs malicioses (com AbuseIPDB) per enriquir la detecció.